📘 Java Notes

---

1. Introduction to Java

- Java is a **high-level, object-oriented, platform-independent** programming language.
- Developed by **Sun Microsystems** (now owned by **Oracle**).
- Known for its slogan: **“Write Once, Run Anywhere (WORA)”** → due to JVM.
- Used for:
    - Desktop Applications
    - Web Applications
    - Mobile Apps (Android)
    - Enterprise Solutions
    - Cloud & Big Data

- ## 2. History of Java
- **1991** → Project started by James Gosling & team (called "Green Project").
- Initially named **Oak** (after an oak tree outside Gosling’s office).
- Later renamed to **Java** (inspired by Java coffee ☕).
- **1995** → Officially released by Sun Microsystems.
- **2009** → Oracle Corporation acquired Sun Microsystems → Java ownership transferred to Oracle.

---

3. Features of Java (with reasons)

Feature	Explanation	Reason
Simple	Syntax similar to C/C++ but no complexity like pointers.	Easy to learn.
Object-Oriented	Everything is in the form of classes & objects.	Promotes reusability & modularity.
Platform-Independent	Code is compiled into **bytecode** which runs on JVM.	Same program runs on any OS.
Secure	No explicit pointers, built-in security APIs.	Prevents unauthorized access.
Robust	Strong memory management, exception handling.	Reduces crashes & errors.
Multithreaded	Can execute multiple tasks simultaneously.	Better performance in concurrent apps.
Portable	Bytecode is platform-independent.	Run anywhere without modification.
High Performance	Uses **JIT (Just-In-Time) compiler**.	Faster execution compared to pure interpretation.

Feature	Explanation	Reason
Distributed	Supports RMI, EJB for distributed computing.	Enables building network-based apps.
Dynamic	Can load classes dynamically at runtime.	More flexible programs.

-

4. What is Package in Java?

- **Definition:** A **package** is a group of related classes, interfaces, and sub-packages.
- Purpose:
  - Avoid name conflicts
  - Reuse code
  - Better organization
- Types:

1. **Built-in Packages** → e.g. java.util, java.io, java.sql
2. **User-defined Packages** → created by programmers.

5. Classes in Java

- **Definition:** A class is a blueprint/template from which objects are created.
- Contains **fields (variables)** and **methods**.

6. Main Method in Java vs C

Aspect	Java	C
Syntax	public static void main(String[] args)	int main()
Return Type	void (does not return value to OS)	int (returns status code to OS)
Entry Point	JVM looks for main method	OS starts execution from main
Arguments	String[] args → command line arguments	int argc, char *argv[]
Modifiers	public static are mandatory	No such modifiers

7. Compiler vs Interpreter

Aspect	Compiler	Interpreter
Definition	Translates entire code into machine code at once.	Translates line by line during execution.
Speed	Faster (once compiled).	Slower (executes line by line).

Aspect	Compiler	Interpreter
Errors	All errors shown after compilation.	Errors shown immediately (line by line).
Example	C, C++	Python, JavaScript
Java Special Case	Java uses **both** → **Compiler (javac)** converts code into **bytecode**, then **JVM's Interpreter + JIT Compiler** executes it.	

1. Arithmetic Operators in Java

- **Definition:** Operators that perform basic mathematical operations.
- **Operators:**
  - + → Addition
  - - → Subtraction
  - * → Multiplication
  - / → Division
  - % → Modulus (remainder)

Example Program:

```
public class ArithmeticDemo {
    public static void main(String[] args) {
        int a = 20, b = 6;

        System.out.println("a + b = " + (a + b));  // Addition
        System.out.println("a - b = " + (a - b));  // Subtraction
        System.out.println("a * b = " + (a * b));  // Multiplication
        System.out.println("a / b = " + (a / b));  // Division
        System.out.println("a % b = " + (a % b));  // Modulus
    }
}
```

Output:

```
a + b = 26
a - b = 14
a * b = 120
a / b = 3
a % b = 2
```

- **Concept:**
  - / gives quotient.
  - % gives remainder.
  - Works on int, float, double etc.

2. Data Types in Java

Java has **two categories of data types**:

(A) Primitive Data Types (8 types)

- Predefined by Java, store simple values (not objects).
- Examples:

Type	Size	Example	Real-life Analogy
byte	1 byte (–128 to 127)	byte age = 25;	Small container (like pen drive of 1GB)
short	2 bytes	short year = 2025;	Year value
int	4 bytes	int salary = 50000;	Employee salary
long	8 bytes	long phone = 9876543210L;	Mobile number
float	4 bytes (decimal up to 7 digits)	float pi = 3.14f;	Approximate values
double	8 bytes (decimal up to 15 digits)	double area = 12345.6789;	Scientific calculations
char	2 bytes (Unicode)	char grade = 'A';	Grades, letters
boolean	1 bit	Boolean is Java Fun = true;	Yes/No situations

(B) Non-Primitive Data Types

- Created by programmers or derived from classes.
- Examples: **String, Arrays, Classes, Interfaces**
- **Real-Life Example:**
  - String → like a sentence ("Hello World")
  - Array → basket holding multiple fruits
  - Class → blueprint of a Car
  - Object → actual Car made from the blueprint

EXAMPLE

```
public class DataTypeDemo {
   public static void main(String[] args) {
      // Primitive
      byte age = 25;
      double price = 99.99;
      boolean isJavaFun = true;

      // Non-Primitive
      String name = "Ajay";
      int[] marks = {90, 85, 80};

      System.out.println("Age: " + age);
      System.out.println("Price: " + price);
```

```
        System.out.println("Is Java Fun? " + isJavaFun);
        System.out.println("Name: " + name);
        System.out.println("First Mark: " + marks[0]);
    }
}
```

3. What is a Variable in Java?

- Definition: A variable is a name given to a memory location to store data.
- Syntax:
- dataType variableName = value;
- Types of Variables in Java:
    1. **Local Variable** → declared inside a method (lives during method execution).
    2. **Instance Variable** → declared inside class but outside method (belongs to object).
    3. **Static Variable** → declared with static keyword (shared by all objects).

java

Copy code

```java
class VariableDemo {
    int instanceVar = 50;     // Instance variable
    static int staticVar = 100; // Static variable

    void show() {
        int localVar = 10;   // Local variable
        System.out.println("Local: " + localVar);
        System.out.println("Instance: " + instanceVar);
        System.out.println("Static: " + staticVar);
    }

    public static void main(String[] args) {
        VariableDemo obj = new VariableDemo();
        obj.show();
    }
}
```

Type of Variable	Declared	Lifetime	Stored	Example
Local	Inside method/block	Until method ends	Stack	int num = 5;
Instance	Inside class, outside methods	As long as object exists	Heap	String name;
Static	Inside class with static keyword	Entire program run	Method Area	static String company;

Variable Type	When to Use
Local	For temporary data used only inside a method/block.
Instance	For object-specific properties (each object has its own copy).
Static	For common/shared properties across all objects.

OOPs vs Procedure-Oriented Programming

Aspect	**OOP (Object-Oriented Programming)**	**POP (Procedure-Oriented Programming)**
Approach	Focuses on **objects** (data + methods).	Focuses on **functions/procedures**.
Example Languages	Java, C++, Python	C, Pascal
Data Security	Data is encapsulated → protected by classes.	Data is global → less secure.
Reusability	Supports **Inheritance & Polymorphism** → high reusability.	Code reuse via functions only.
Structure	Divides program into **objects**.	Divides program into **functions**.
Real-life analogy	Blueprint (class) → House (object).	Step-by-step recipe (procedures).

Compiler vs Interpreter (and how they work together in Java)

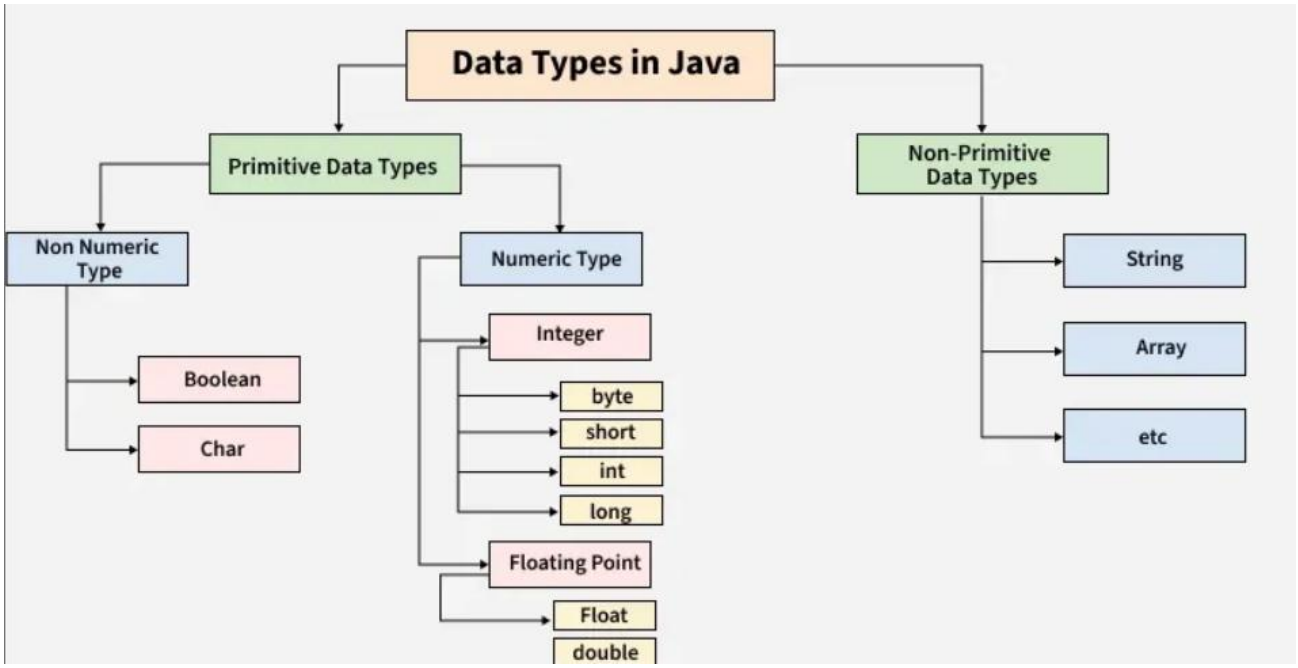
Aspect	Compiler	Interpreter
Definition	Translates **entire source code** into machine code at once.	Translates **line by line** at runtime.
Speed	Faster execution after compilation.	Slower (line-by-line).
Error Detection	Shows all errors at once after compilation.	Stops immediately when error occurs.
Example	C, C++	Python, JavaScript

In Java → Both are used together:

1. **Compiler (javac)** → Converts .java file → .class file (bytecode).
2. **JVM (Java Virtual Machine)** → Uses **Interpreter + JIT (Just-In-Time Compiler)** to convert bytecode → machine code.

✅ This is why Java is **platform-independent**.

Data Types in Java (Flow Chart)



Syntax to Create an Object in Java

ClassName objectName = new ClassName();

Common Methods in Java

(A) String Methods

- length() → returns length of string
- toUpperCase(), toLowerCase()
- charAt(int index)

- substring(int begin, int end)
- equals(), equalsIgnoreCase()

```
String str = "Java";
System.out.println(str.length());       // 4
System.out.println(str.toUpperCase()); // JAVA
```

(B) Wrapper Class Methods

- Integer.parseInt("123") → converts String → int
- String.valueOf(123) → converts int → String

(C) Math Class Methods

- Math.sqrt(25) → 5.0
- Math.pow(2,3) → 8.0
- Math.max(10,20) → 20
- Math.random() → random double [0.0 – 1.0)

(D) Object Class Methods (superclass of all classes)

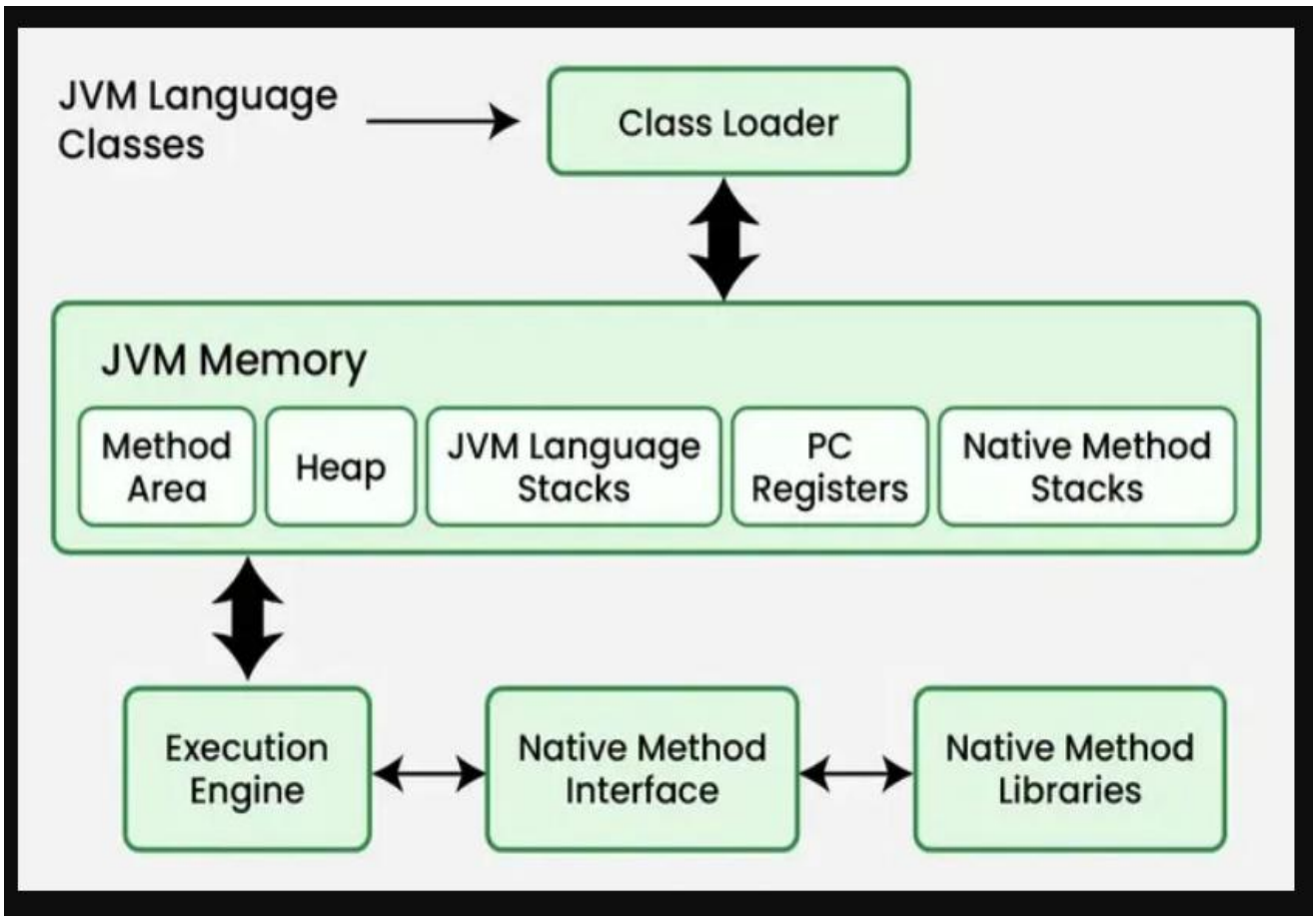
- toString() → returns string representation
- equals(Object obj) → compares objects
- hashCode() → returns hash code

☕ Java Environment Components

1. JVM (Java Virtual Machine)

- **Definition:** JVM is an abstract machine that runs **Java bytecode** (.class files).
- **Key Functions:**
    1. **Loads code** → Class Loader loads .class files into memory.
    2. **Verifies code** → Bytecode verifier ensures no illegal code (security).
    3. **Executes code** → Interpreter + JIT (Just-In-Time) Compiler convert bytecode → machine code.
    4. **Manages Memory** → Uses **Heap** (for objects), **Stack** (for methods), and **Garbage Collector** (to clean unused objects).

- **JVM Architecture (Simplified)**



2. JRE (Java Runtime Environment)

- **Definition:** Software package that provides environment to **run Java applications**.
- **Includes:**
  - **JVM**
  - **Core Libraries** (like java.lang, java.util, java.io)

- **Does Not Include:** Compiler (javac) → cannot **develop** Java programs.

👉 Use **JRE** when you just want to **run** a Java program.

Example:
If you download a Java game or desktop app → you only need JRE.

---

3. JDK (Java Development Kit)

- **Definition:** Full software package for **developing + running** Java programs.
- **Includes:**
  - **JRE** (JVM + libraries)
  - **Development Tools:**
    - javac → Java Compiler (compiles .java → .class)
    - java → Launcher (runs program)
    - javadoc → Generates documentation
    - jdb → Debugger

👉 Use **JDK** when you want to **write, compile, and run** Java programs.

4. Relationship Between JDK, JRE, JVM

JDK = JRE + Development Tools

JRE = JVM + Libraries

JVM = Executes Bytecode

Or visually:

JDK

└── JRE

└── JVM

5. Example (Step-by-step Execution in Java)

1. Write program → Hello.java

```
class Hello {
   public static void main(String[] args) {
      System.out.println("Hello World");
   }
}
```

2. Compile using **JDK Compiler**:

```
javac Hello.java
```

→ Produces Hello.class (bytecode).

3. Run using **JRE (java command → JVM executes)**:

java Hello

→ Output:

Hello World

---

6. Key Interview Points

- **JVM**: Machine-dependent (different JVMs for Windows, Linux, Mac).
- **Bytecode**: Machine-independent (same .class runs everywhere).
- **JRE**: Only runs programs, cannot compile.
- **JDK**: Contains everything → used by developers.
- **HotSpot JVM**: Default JVM used by Oracle.
- **JIT Compiler**: Improves performance by compiling repeated bytecode into native code.

Decision-Making Statements in Java

- Used to control program flow based on conditions.

(A) if statement

java Copy code

```java
int age = 18;
if (age >= 18) {
    System.out.println("You are eligible to vote.");
}
```

(B) if-else statement

java Copy code

```java
int number = 5;
if (number % 2 == 0) {
    System.out.println("Even");
} else {
    System.out.println("Odd");
}
```

(C) if-else-if ladder

java

Copy code

```java
int marks = 75;
if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 75) {
    System.out.println("Grade B");
} else if (marks >= 50) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}
```

(D) Nested if

java

Copy code

```java
int age = 25;
int weight = 60;
if (age >= 18) {
    if (weight > 50) {
        System.out.println("Eligible for donation");
    }
}
```

(E) switch statement

java Copy code

```java
int day = 3;
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    case 3: System.out.println("Wednesday"); break;
    default: System.out.println("Invalid Day");
}
```

🔁 Looping Statements in Java

👉 Definition:

A loop is used to execute a block of code **repeatedly** until a condition is true.

1. for loop

- Used when the **number of iterations is known**.
- **Syntax:**

java Copy code

```java
for(initialization; condition; update) {
    // code
}
```

```java
public class ForLoopDemo {
    public static void main(String[] args) {
        for(int i = 1; i <= 5; i++) {
            System.out.println("Count: " + i);
        }
    }
}
```

Output:

```makefile
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

2. while loop

- Used when the **number of iterations is not fixed**.
- **Syntax:**
-

```java
while(condition) {
    // code
}
```

java Copy code

```java
public class WhileLoopDemo {
    public static void main(String[] args) {
        int i = 1;
        while(i <= 5) {
            System.out.println("Hello " + i);
            i++;
        }
    }
}
```

3. do-while loop

- Similar to while, but **executes at least once** (condition checked later).
- **Syntax:**

java Copy code

```java
do {
    // code
} while(condition);
```

java Copy code

```java
public class DoWhileDemo {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println("Number: " + i);
            i++;
        } while(i <= 5);
    }
}
```

4. for-each loop (Enhanced for loop)

- Used to **iterate over arrays or collections**.
- **Syntax:**

java Copy code

```
for(dataType var : array) {
    // code
}
```

java Copy code

```
public class ForEachDemo {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40};
        for(int num : numbers) {
            System.out.println(num);
        }
    }
}
```

Output:

Copy code

```
10
20
30
40
```

Key Points

- for → fixed iterations.
- while → condition-based (unknown iterations).
- do-while → runs at least once.
- for-each → best for arrays/collections.

🔁 Difference Between Loops in Java

Loop Type	When to Use	Condition Checking	Executes At Least Once?	Example Use Case
for loop	When number of iterations is **known in advance**.	Checked **before** each iteration.	❌ No (may not run if condition false initially).	Printing 1 to 10 numbers.
while loop	When number of iterations is **unknown**, condition-based.	Checked **before** each iteration.	❌ No (may not run if condition false initially).	Read data until user presses "exit".
do-while loop	When code must execute **at least once**, then check condition.	Checked **after** execution.	✅ Yes (runs at least once).	Menu-driven program (ATM, game).
for-each loop	When iterating through **arrays or collections**.	Automatically checks till end of collection.	❌ No (depends on elements).	Printing elements of array.

Copy code

```java
public class LoopComparison {
    public static void main(String[] args) {
        // for loop
        for(int i=1; i<=3; i++) {
            System.out.println("for loop: " + i);
        }

        // while loop
        int j = 1;
        while(j <= 3) {
            System.out.println("while loop: " + j);
            j++;
        }

        // do-while loop
        int k = 1;
        do {
            System.out.println("do-while loop: " + k);
            k++;
        } while(k <= 3);
```

```java
    // while loop
    int j = 1;
    while(j <= 3) {
        System.out.println("while loop: " + j);
        j++;
    }

    // do-while loop
    int k = 1;
    do {
        System.out.println("do-while loop: " + k);
        k++;
    } while(k <= 3);

    // for-each loop
    int[] arr = {10, 20, 30};
    for(int num : arr) {
        System.out.println("for-each loop: " + num);
    }
}
```

✅ Summary:

- **for** → fixed count.
- **while** → uncertain count, condition-based.
- **do-while** → ensures one-time execution.
- **for-each** → best for arrays/collections.